Design Analysis Report: ExamEnv

Session 8: Software Design Principles. Modularization, coupling and cohesion analysis, function-oriented vs object-oriented design, top-down and bottom-up design. Industry mapping: Software Design Engineering.

Project: ExamEnv, Infrastructure-as-Code for Exams. Group 1, Software Engineering / SDP – CA-2.

Team member	Role
Gaurav Shukla	CloudOps and Agents
Gaurav Singh	Backend and Research
Manya Tiwari	Backend and Research
Nandani	ML (Parser)

1. Introduction

ExamEnv's design is modular, loosely coupled and mostly cohesive, with a few specific weak spots around the shared environment spec and two overloaded classes. ExamEnv turns a teacher's requirement PDF into a reviewed YAML spec, lets each student provision an identical Docker environment with one command, and shows the teacher live readiness for the whole class.

This report analyses that design using the team's earlier work: the 14 use cases and narratives from Session 5 (UC01 to UC14) and the 17-class model from Session 6, plus the future-release Template class. It covers modularization, coupling and cohesion, function-oriented versus object-oriented design, and top-down versus bottom-up design, and ends with concrete improvements.

The order of interactions between modules is taken from the main flows in the use case narratives (UC02 to UC12).

2. System Overview

ExamEnv has four zones. Each zone realises a group of use cases and owns a group of classes.

Zone	Component	Use cases	Main classes
1	Teacher Dashboard	UC02, UC04, UC05, UC06, UC07, UC14	Teacher, Exam, RequirementDocument, EnvironmentSpec
2	Backend Hub (FastAPI, PostgreSQL, Celery/Redis)	UC01, UC13, UC05, UC12	User, Admin, Exam, ParseJob, StatusRecord, VerificationLog, StatusBroadcaster
3	ML Parser	UC03	MLParser

Zone	Component	Use cases	Main classes
4	Student Agent + Docker Engine	UC08, UC09, UC10, UC11, UC12	Student, StudentAgent, EnvironmentInstance, StatusRecord, VerificationLog

The actors are Teacher, Student and Admin (primary), and the ML Parser (LLM service) and Docker Engine (secondary). The data flow has three phases: upload and parsing (UC02 to UC05), provisioning (UC08 to UC12) and monitoring (UC06 and UC07).

3. Design Principles Applied

Eight principles are visible in the Session 5 and 6 models.

Principle	Where it shows in ExamEnv
Separation of concerns	Four zones. Upload (UC02) and parsing (UC03) are separate use cases, so the teacher is never blocked while the parser runs.
Single responsibility	ParseJob is separate from RequirementDocument, and StatusRecord is separate from VerificationLog (Session 6 design decisions). Each tracks one thing.
Abstraction	EnvironmentSpec is built from PackageSpec, PortMapping and VolumeMount, so YAML can be validated field by field.
Information hiding	StudentAgent hides Docker behind EnvironmentInstance (buildImage, runContainer). MLParser keeps its promptTemplate private, so no other class knows which prompt or model is used.
Inheritance for reuse	Teacher, Student and Admin inherit login, logout and password handling from User.
Open/Closed	UC14 (Template Library) extends UC02, and Template only seeds a draft EnvironmentSpec, so the existing flow does not change when templates are added.
Human in the loop	EnvironmentSpec moves from DRAFT to PUBLISHED only after the teacher reviews it (UC04, UC05).
Least surprise	UC09 shows a dry-run preview and asks for confirmation before anything is installed.

4. Modularization

ExamEnv is split into 11 modules that follow the repository folders and group the Session 6 classes by use case. Three criteria drove the split:

1. **Change axis.** Parts that change for different reasons are apart: LLM prompts change often, Docker lifecycle code rarely.

2. **Runtime boundary.** Parts that run in different places are apart: teacher's browser, server, student's laptop.
3. **Ownership.** Each team member owns one subsystem: Nandani the ML Parser, Gaurav Shukla the agent and Docker side, Gaurav Singh and Manya Tiwari the backend.

4.1 Module catalogue

Module (folder)	Classes	Use cases	Responsibility
backend/app/api (accounts)	User, Teacher, Student, Admin	UC01, UC13	Registration, login, tokens, roles
backend/app/api (exams and specs)	Exam, RequirementDocument, EnvironmentSpec, PackageSpec, PortMapping, VolumeMount	UC02, UC04, UC05	Upload, spec CRUD, validation, publish
backend/app/api (status)	StatusRecord, VerificationLog	UC07, UC12	Receive agent reports, serve logs
backend/app/models	All persistent entities	All	PostgreSQL tables: Users, Specs, Status, Logs
backend/app/tasks	ParseJob	UC03	Queue, run and retry parsing
backend/app/websockets	StatusBroadcaster	UC06	Push live status to dashboards
ml-parser/parser	MLParser	UC03	Extract text and requirements, generate YAML
ml-parser/templates	Template (future)	UC14	Pre-built verified specs
student-agent/agent	StudentAgent	UC08, UC09, UC12	CLI: pull, dry-run, report
student-agent/docker	EnvironmentInstance	UC10, UC11	Build image, run container, verify
frontend/dashboard	Screens for Teacher	UC02, UC04, UC05, UC06, UC07	Upload, YAML editor, live status grid

4.2 Dependency view

```
Teacher Dashboard --REST/WS--> Backend Hub --Celery job--> ML Parser
                                    ^
                                    | REST (spec, status report)
```

```
                                  |
Student Agent --builds/runs--> Docker Engine
```

All dependencies point to the Backend Hub and to the EnvironmentSpec. The dashboard and the agent never call each other, and the agent never calls the parser.

4.3 Assessment

- **Strengths:** every use case maps to one module group, so a change request touches few modules. The parser can be replaced without touching the agent. The agent can be tested with a mocked backend.
- **Weakness:** four modules (exams and specs, tasks, parser, agent) depend on the shape of EnvironmentSpec, but no module owns that shape (Section 9, item 1).

5. Coupling Analysis

Coupling is low to medium overall, and the class diagram has no circular dependencies. The scale used, from best to worst, is: data, stamp, control, common, content. Each row below is a relationship from the Session 6 class diagram.

#	Classes	Relationship	Coupling	Level	Reasoning
1	Teacher, Student, Admin to User	Inheritance	Class inheritance	Medium	Subclasses depend on User internals. Kept to one level, so it is manageable.
2	Exam and RequirementDocument	Composition	Data	Medium	Documents are removed with the exam. Intentional lifecycle link.
3	ParseJob to MLParser	Dependency	Data	Low	One-way. Job passes text and gets requirements back. MLParser does not know ParseJob.
4	ParseJob and EnvironmentSpec	Association 1 to 1	Stamp	Low to Medium	Job produces a draft spec and shares its structure.
5	Exam and EnvironmentSpec	Aggregation	Data	Low	Specs can exist independently of the exam.
6	EnvironmentSpec and its parts (PackageSpec, PortMapping, VolumeMount)	Composition	Stamp	Medium	Parts exist only inside a spec. Tight by design, justified by high cohesion.
7	EnvironmentSpec and EnvironmentInsta	Association 1 to many	Stamp	Medium	The instance is built from spec fields. This is the most shared structure in the

#	Classes	Relationship	Coupling	Level	Reasoning
	nce				system.
8	StudentAgent to EnvironmentInstance	Dependency	Data	Low	Agent creates and controls instances through methods only.
9	EnvironmentInstance and StatusRecord, VerificationLog	Composition	Data	Low to Medium	Status and logs belong to one instance.
10	StatusBroadcaster to StatusRecord	Dependency	Data (event)	Low	Broadcaster only publishes changes.
11	Teacher to StatusBroadcaster	Dependency	Data	Medium	A domain class depends on a transport mechanism (WebSocket). See Section 9, item 2.
12	Template to EnvironmentSpec	Dependency (future)	Data	Low	One-way seeding of a draft.

Findings

- **No content coupling.** Classes interact through methods, and the REST API separates the dashboard, backend and agent.
- **EnvironmentSpec is the hotspot.** Exam, ParseJob, Template and EnvironmentInstance all depend on it, and the agent and parser rely on its YAML structure. A field rename ripples through four modules.
- **Teacher to StatusBroadcaster (row 11)** is the one dependency that points the wrong way: an account class knows about the real-time delivery mechanism.

6. Cohesion Analysis

Most ExamEnv classes are functionally cohesive; the lower scores are the orchestrators and two classes that mix concerns. The scale, from best to worst, is: functional, sequential, communicational, procedural, temporal, logical, coincidental.

Class	Methods	Cohesion	Level	Comment
ParseJob	run, retry, markFailed	Functional	High	One job lifecycle.
EnvironmentInstance	buildImage, runContainer, stop, cleanup	Functional	High	Container lifecycle only.

Class	Methods	Cohesion	Level	Comment
StatusRecord	update, toJson	Functional	High	One status state.
VerificationLog	record	Functional	High	One check result.
StatusBroadcaster	subscribe, broadcast, unsubscribe	Functional	High	Only WebSocket fan-out.
PackageSpec, VolumeMount, Template	toInstallCommand, describe, instantiate	Functional	High	One small job each.
PortMapping	isAvailable	Functional	Medium to High	Checks the student's machine, which is a runtime concern inside a spec part.
User, Student, Admin	register, login, createUser, and similar	Functional	High	Account actions only.
MLParser	extractText, extractRequirements, generateYaml	Sequential	Medium to High	The output of each step feeds the next (UC03 pipeline).
RequirementDocument	store, triggerParsing	Communicational	Medium	Storing a file and starting a job are different concerns.
Exam	create, getPublishedSpec, getClassStatus	Communicational	Medium	getClassStatus reaches into status data.
EnvironmentSpec	validate, edit, publish, toDockerfile	Communicational	Medium to High	The first three work on spec content. toDockerfile adds Docker knowledge.
Teacher	uploadRequirement, reviewSpec, editSpec, publishSpec, viewClassStatus, viewErrorLog	Communicational	Medium	Six workflow actions across exams, specs and status.
StudentAgent	fetchExams, pull, dryRun, provision,	Procedural	Medium	Sequences UC08 to UC12. Expected for an

Class	Methods	Cohesion	Level	Comment
	verify, report			orchestrator, as long as it stays thin.

Findings

- 10 of the 17 classes score high, and none is coincidental or logical.
- Teacher and EnvironmentSpec are the two classes to improve: Teacher carries workflow logic that belongs in services, and EnvironmentSpec holds Docker-specific code (toDockerfile).

7. Function-Oriented vs Object-Oriented Design

ExamEnv uses a hybrid: an object-oriented model for entities and state, and function-oriented pipelines for the use cases that transform data step by step.

Aspect	Function-oriented	Object-oriented
Primary unit	Functions and data flow	Objects with state and behaviour
Focus	What the system does	What the system is made of
State	Passed between functions	Encapsulated in objects
Reuse	Function libraries	Inheritance, composition, polymorphism
Best for	Pipelines and transformations	Domain entities and long-lived state

7.1 Where each style is used

Part of ExamEnv	Style	Why
Domain model (Session 6, 17 classes)	Object-oriented	Entities with identity and state: Exam, EnvironmentSpec, EnvironmentInstance.
User hierarchy	Object-oriented (inheritance)	Teacher, Student and Admin share login code from User.
EnvironmentSpec and its parts	Object-oriented (composition)	Parts exist only inside a spec and validate themselves.
State machines	Object-oriented (enumerations)	Role, JobState, SpecState and ProvisionState control what each object may do next.
PDF to YAML (UC03)	Function-oriented	extractText, extractRequirements, generateYaml and validate form a pipeline where each output feeds the next.
Provisioning flow (UC08 to UC12)	Function-oriented	pull, dryRun, provision, verify and report run in a fixed order.

Part of ExamEnv	Style	Why
Async parsing	Function-oriented	A Celery task is a function that takes a job id and returns a result.

7.2 Decision and a gap

A pure OO design would wrap simple transformations in needless classes. A pure function-oriented design would scatter rules such as spec validation and state transitions. The hybrid keeps rules next to the data they protect and keeps pipelines easy to test as plain functions.

One gap remains: UC11 (Verify Environment) is a set of different checks (versions, pip list, imports, ports). VerificationLog already records checkName, command, output and passed, but there is no class for the checks themselves. A Check interface with one subclass per check type would let new checks be added without changing the verifier (Open/Closed).

8. Top-Down and Bottom-Up Design

The architecture was derived top-down from the use cases, and the small reusable parts were identified bottom-up from the noun-verb analysis and the verification scripts.

8.1 Top-down decomposition

```
Level 0  ExamEnv: automate exam environment setup
|
Level 1  +-- Access and accounts   (UC01, UC13)
|        +-- Spec authoring        (UC02, UC03, UC04, UC05, UC14)
|        +-- Provisioning          (UC08, UC09, UC10, UC11, UC12)
|        +-- Monitoring            (UC06, UC07)
|
Level 2  Authoring:    upload PDF -> parse -> review/edit -> publish
|        Provisioning: pull -> dry-run -> provision -> verify -> report
|        Monitoring:   live status grid -> error log
|
Level 3  UC03 -> MLParser.extractText -> extractRequirements -> generateYaml ->
EnvironmentSpec.validate
         UC10 -> EnvironmentInstance.buildImage -> runContainer
         UC11 -> verification checks -> VerificationLog.record
```

The goal and the three project phases were fixed first (Sessions 4 and 5). Each phase was then split into use cases, and the use cases were refined into classes and methods (Session 6). This made it possible to hand each subsystem to one team member against agreed interfaces.

8.2 Bottom-up elements

- **Small spec parts.** PackageSpec.toInstallCommand, PortMapping.isAvailable and VolumeMount.describe are tiny, independently testable units later assembled into EnvironmentSpec.
- **Verification scripts.** Version, pip list and import checks are simple standalone commands that UC11 generalises and runs against the spec.

- **Dockerfile generation.** Turning a package list into a Dockerfile can be built and tested before any backend exists.
- **Template Library.** Pre-built YAML specs for common courses (Python, Web Dev, Databases, ML) are standalone assets that seed drafts (UC14) and can double as test fixtures.
- **Noun-verb analysis.** Session 6 derived classes from nouns and verbs in the SRS, and these were then checked against the use cases.

8.3 How the two combine

Approach	Used for	Benefit
Top-down	Architecture, phases, module boundaries, REST contracts	The parts fit together and every use case is covered.
Bottom-up	Spec parts, verification scripts, Dockerfile generator, templates	Early, testable, reusable pieces with lower integration risk.

Contracts are defined top-down, low-level parts are built and tested bottom-up, and the two meet at the EnvironmentSpec.

9. Design Issues and Recommendations

Eight issues were found; the first two have the largest effect on coupling and cohesion.

#	Issue	Evidence in the models	Recommendation
1	EnvironmentSpec structure is an implicit shared contract.	Used by Exam, ParseJob, Template, EnvironmentInstance, and by the parser and agent through YAML. UC03 validates against a schema, but no module owns it.	Define one versioned schema (for example a Pydantic or JSON Schema model) in a shared package and validate at every boundary.
2	Teacher carries workflow logic and depends on StatusBroadcaster.	Six workflow methods, plus a dependency on the WebSocket mechanism.	Keep Teacher as an account type. Move workflows to services (for example SpecService, StatusService) and let the dashboard subscribe through the API.
3	EnvironmentSpec.toDockerfile mixes spec data with Docker knowledge.	EnvironmentSpec lists toDockerfile beside validate, edit and publish.	Move Dockerfile generation to a DockerfileBuilder in student-agent/docker. The spec stays pure data and rules.
4	PortMapping.isAvailable checks runtime state on the student	UC09 exception E1 (port already in use) is an agent preflight check.	Move the availability check into the dry-run step (UC09) and keep PortMapping as data.

#	Issue	Evidence in the models	Recommendation
	machine.		
5	RequirementDocument .triggerParsing couples a stored file to job creation.	UC02 step 5 queues the parse job.	Let the upload service create the ParseJob after store succeeds.
6	StudentAgent sequences five steps (procedural cohesion).	pull, dryRun, provision, verify, report in one class.	Keep it a thin orchestrator that only calls other modules, with no business rules.
7	Verification checks are not extensible.	UC11 runs several check types, but only the log has a class.	Add a Check interface with one subclass per check type, for example version, package, import and port.
8	Poor connectivity in exam rooms and status integrity.	UC12 alternate flow resends when the network returns. StudentAgent already holds authToken.	Cache the pulled spec locally, authenticate every report with the token, and make report posting idempotent per student and exam.

10. Summary and Conclusion

ExamEnv's design meets the Session 8 principles well, and implementing the recommendations in Section 9, especially the shared spec schema and a lighter Teacher class, would make it stronger.

Criterion	Result
Modularization	High. 11 modules, every use case mapped to a module group, one owner per subsystem.
Coupling	Low to medium, no circular or content coupling. Hotspots: EnvironmentSpec (stamp) and Teacher to StatusBroadcaster.
Cohesion	10 of 17 classes high. Teacher and EnvironmentSpec are medium. StudentAgent is a procedural orchestrator.
Paradigm	Hybrid: object-oriented domain model and state machines, function-oriented pipelines (UC03, UC08 to UC12).
Design process	Top-down from use cases to classes, bottom-up for spec parts, verification scripts, Dockerfile generation and templates.
Priority fixes	Shared versioned YAML schema, move workflows out of Teacher, move toDockerfile out of EnvironmentSpec, add a Check interface.

The design divides the system into modules that follow the use cases, keeps most dependencies as simple data exchanges, and gives most classes a single purpose. These choices keep future additions, such as the Template Library, offline mode and new input formats, local to a few modules.